

Nivell 1: Entorn i Ingesta Híbrida (Code-First)

Escenari:

El quadre de comandament financer tarda massa a carregar i el cost és inacceptablement alt. S'ha identificat que el problema crític són els informes que creuen vendes amb regions geogràfiques.

Exercici 1: Consulta sobre Taula no Optimitzada (Diagnòstic)

El Country Manager d'**Alemanya** necessita revisar urgentment les transaccions del dia **12 de març de 2022**.

Tasca:

- » 1. Escriu la consulta que uneix (JOIN) transaccions i companyies.
- » 2. Filtra els resultats per la data indicada i el país "Germany".
- » 3. Sense executar la consulta, realitza un "Dry Run" (auditoria de costos).

 *Observació: Fixa't que BigQuery llegeix gairebé tota la taula tot i demanar només un dia (Full Table Scan).*

Utilizo ON en vez de USING ya que la columna tiene un nombre distinto en ambas tablas.

Al hablar de ventas cruzadas, decido añadir declined = 0 para garantizar que son transacciones realizadas.



```
1 SELECT *
2 FROM sprint3_silver.transactions_clean AS tc
3 JOIN sprint3_silver.companies_clean AS cc
4   ON(tc.business_id = cc.company_id)
5 WHERE 1=1
6    AND tc.declined = 0
7    AND DATE(tc.timestamp) = DATE '2022-03-12'
8    AND cc.country = 'Germany';
9
10
```

Esta consulta procesará 11.86 MB cuando se ejecute.

Nota para mi: también se puede resolver sin aplicar una función en la columna, con un intervalo tal que así:

```
WHERE tc.timestamp >= TIMESTAMP('2022-03-12 00:00:00 UTC')
AND tc.timestamp <  TIMESTAMP('2022-03-13 00:00:00 UTC')
```

Exercici 2: Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster)

Escenari:

Després del diagnòstic de l'exercici anterior, hem confirmat que filtrar la taula de transaccions per data o per empresa provoca un *Full Table Scan* (lectura completa de la taula), cosa que és ineficient. Per solucionar-ho a la capa de consum (*Gold*), materialitzarem una versió optimitzada de la taula de fets.

Problema del sandbox: com que utilitzem un entorn gratuït sense targeta de crèdit, BigQuery esborra automàticament les dades de les particions amb més de 60 dies d'antiguitat. Atès que les nostres dades històriques són antigues, si particionem directament la taula, aquesta acabarà quedant buida. Per evitar-ho, ho resoldrem en dos passos: primer, actualitzarem automàticament les dates de les dades i, després, aplicarem l'optimització física.

Tasca:

Pas 1: Generació de Dades Recents (Mocking Data)

Crea una taula intermèdia anomenada `sprint3_silver.transactions_recent` a partir de la taula `sprint3_silver.transactions_clean`. El teu objectiu és mantenir totes les columnes, però substituir el timestamp original per un de nou, generat aleatòriament perquè caigui dins dels últims 50 dies.

» **Pista Tècnica:** Per fer-ho sense escriure totes les columnes a mà, pots utilitzar `SELECT * EXCEPT(timestamp)`. Per calcular la data aleatòria, hauràs de restar una quantitat de dies a l'instant actual. Et seran molt útils les funcions `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`, i la combinació `CAST(RAND() * 50 AS INT64)` per generar els dies de desfasament.

Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering)

Ara, crea la teva taula definitiva `sprint3_gold.fact_transactions_optimized` a partir de les dades recents que acabes de generar a `transactions_recent`. Has de construir la sentència DDL per configurar la taula amb les següents estratègies físiques d'emmagatzematge:

» **Particionament (Partitioning):** Divideix la taula per la data del camp `DATE(timestamp)`. Això permetrà que les consultes que filtrin per dia (`WHERE date = ...`) només llegeixin la partició necessària.

» **Clustering:** Ordena les dades dins de cada partició per business_id. Això accelerarà dràsticament els filtres per companyia i els encreuaments (JOINS) amb la dimensió d'empreses.

👉 **Entrega:**

- » El codi SQL utilitzat per al Pas 1.
- » El codi DDL (CREATE OR REPLACE TABLE...) utilitzat al Pas 2.
- » Una captura de pantalla dels "Detalls" de la nova taula fact_transactions_optimized on es pugui comprovar que està particionada i agrupada en clústers correctament.

PAS 1

```
CREATE OR REPLACE TABLE sprint3_silver.transactions_recent AS
SELECT * EXCEPT(`timestamp`),
TIMESTAMP_SUB(
  CURRENT_TIMESTAMP(),
  INTERVAL CAST(RAND() * 50 AS INT64) DAY) AS `timestamp`
FROM `sprint3_silver.transactions_clean`;
```

PAS 2

```
CREATE OR REPLACE TABLE sprint3_gold.fact_transactions_optimized
PARTITION BY DATE(timestamp)
CLUSTER BY business_id
AS
SELECT *
FROM sprint3_silver.transactions_recent;
```

Esquema		Detalles	Vista previa	Ex
Etiquetas				
Tipo de tabla	Particionada			
Particionada por	DAY			
Particionada en el campo	timestamp			
Vencimiento de la partición	Las particiones no vencen			
Filtro de partición	No es obligatorio			
Agrupada por	business_id			

Exercici 3: La Prova del Cotó (Benchmark)

Escenari:

Escenari: Ja has creat la taula `sprint3_gold.fact_transactions_optimized` amb les estratègies de Particionament i Clustering. Ara toca demostrar a l'equip la millora de rendiment i l'estalvi de costos que has aconseguit. Simularem una petició típica de negoci: "Volem veure totes les transaccions dels últims 30 dies".

✂ Tasca:

L'objectiu és clar i directe: **aplicar exactament la mateixa consulta a dues taules diferents i comparar-ne el cost computacional.**

Construeix una consulta SQL que seleccioni totes les columnes (`SELECT *`) i filtri les dades dels últims 30 dies.

⚠ **NO executis les consultes encara.** En aquest exercici només utilitzarem el validador de BigQuery ("Dry Run", a dalt a la dreta de l'editor) per visualitzar els **Bytes processats** de manera gratuïta.

Fes la comparativa seguint aquests dos passos:

- » **Pas 1 (Taula no optimitzada):** Apunta la teva consulta a la taula `sprint3_silver.transactions_recent`. Observa el validador i fes una captura de pantalla dels bytes que processaria.
- » **Pas 2 (Taula optimitzada):** Sense canviar res de la lògica de la teva consulta, modifica només el `FROM` perquè ara apunti a la taula particionada `sprint3_gold.fact_transactions_optimized`. Observa com canvia el validador i fes la segona captura.

👉 Entrega:

- » El codi SQL base que has dissenyat.
- » Les dues captures de pantalla on es vegi la diferència de "Bytes processats" al validador.
- » Un breu text calculant quin és el **% d'estalvi** que has aconseguit (compara els bytes del Pas 1 respecte als del Pas 2).

```
SELECT *
FROM sprint3_silver/gold.fact_transactions_optimizez/transactions_recent
WHERE DATE(timestamp) BETWEEN DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
AND CURRENT_DATE();
```

Opto por esta opción por su legibilidad y dinamismo. También porque incluye ambos extremos (hoy y hace 30 días).

De manera similar, algo más larga, podríamos cambiar el BETWEEN por (> o >= combinado con < o <=, en función de si queremos incluirlos o no).

Descarto contar “manualmente” 30 días hacia atrás ya que poner una fecha dinámica limita, y priorizo el dinamismo.

Dry run para la tabla en silver vs Dry run para la tabla en gold:

The image displays two side-by-side screenshots of a SQL query editor, likely BigQuery, showing SQL queries and their dry run results.

Left Screenshot (Silver Table):

```
1 SELECT *
2 FROM sprint3_silver.transactions_recent
3 WHERE 1=1
4 AND DATE(timestamp) BETWEEN DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
5 AND CURRENT_DATE();
6
```

Below the query, a status bar indicates: "Esta consulta procesará 11.85 MB cuando se ejecute."

Right Screenshot (Gold Table):

```
1 SELECT *
2 FROM sprint3_gold.fact_transactions_optimized
3 WHERE 1=1
4 AND DATE(timestamp) BETWEEN DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
5 AND CURRENT_DATE();
6
```

Below the query, a status bar indicates: "Esta consulta procesará 6.75 MB cuando se ejecute."

Ahorro:

La consulta sobre la 2a tabla (la optimizada) procesaría 6,75 MB, mientras que la original, 11.85 MB, según el Dry run.

Utilizar la segunda tabla supone una reducción de 5,10 MB, es decir, un ahorro aproximado del 43,04%.

Este resultado apoya que la optimización mediante *partitioning* y *clustering* reduce la cantidad de datos que BigQuery procesará, mejorando el rendimiento y disminuyendo los costes, lo que se traduce en mayor eficiencia.

Exercici 4: Smart Caching (Vistes Materialitzades)

Escenari:

El Director General té un quadre de comandament que mostra les "**Vendes Totals per Dia**". Ell (i 50 managers més) refresquen aquest gràfic constantment.

Problema:

Si utilitzes una consulta normal o una Vista estàndard, cada vegada que algú refresca la pàgina, BigQuery recalcula la suma de milions de files. És lent i estàs pagant pel mateix càlcul una vegada i una altra.

Solució:

Utilitzar una **Vista Materialitzada**. Aquesta tecnologia guarda el *resultat* del càlcul a la memòria cau i només l'actualitza si entren dades noves (incremental).

Tasca:

- » Crea una **Vista Materialitzada** anomenada `sprint3_gold.mv_daily_sales` que mostri les **Vendes Totals per Dia**.

Entrega:

- » Codi per Crear la **Vista Materialitzada**.
- » Fes una consulta que mostri la vista creada i fes una captura de pantalla que mostri en el seu conjunt la vista materialitzada creada, amb la consulta executada i els bits processats.

```
CREATE OR REPLACE MATERIALIZED VIEW sprint3_gold.mv_daily_sales AS
SELECT DATE(tc.timestamp) AS Data, SUM(tc.amount) AS Vendes_Totals_Dia
FROM sprint3_silver.transactions_clean AS tc
GROUP BY DATE(tc.timestamp);
```

The screenshot shows the Google Cloud BigQuery interface. On the left, the 'Conexiones' (Connections) sidebar lists the project 'sprint3-gold' and its datasets, including 'mv_daily_sales'. The main panel displays a query titled 'Consulta sin titulo' with the following SQL code:

```
1 CREATE OR REPLACE MATERIALIZED VIEW sprint3_gold.mv_daily_sales AS
2 SELECT DATE(tc.timestamp) AS Data, SUM(tc.amount) AS Vendes_Totals_Dia
3 FROM sprint3_silver.transactions_clean AS tc
4 GROUP BY DATE(tc.timestamp);
```

The query has been executed successfully, as indicated by the green checkmark and the message 'Se completó la consulta'. Below the query, the 'Resultados de la consulta' (Query Results) section is visible, showing tabs for 'Información del trabajo', 'Resultados', 'Detalles de la ejecución', and 'Gráfico de ejecución'. A notification at the bottom states: 'Esta declaración creó una nueva vista materializada denominada mv_daily_sales.' with a button to 'Ir a la vista materializada'.

Esquema

Detalles

Estadísticas

Linaje

Perfil de datos

Calidad de los datos

Filtro

Escribir el nombre o valor de la propiedad

?

☐	Nombre del campo	Tipo	Modo	Descripción	Clave	Intercalación	Valor pred
☐	Data	DATE	NULLABLE	-	-	-	-
☐	Vendes_Totals_Dia	FLOAT	NULLABLE	-	-	-	-

Nivell 2: SQL Analític Avançat

Escenari:

La direcció comença a fer preguntes que no es resolen amb un simple `GROUP BY`. Necessitem anàlisi de tendències i comportament d'usuari.

Exercici 1: Perfilat de Clients VIP (Mètriques Agregades amb CTEs)

Escenari:

L'equip de Màrqueting vol analitzar el comportament dels nostres millors clients per dissenyar l'estratègia del pròxim any. Definim "VIP" com aquells amb una despesa acumulada superior a 500€. Necessiten un informe que, per a cada VIP, mostri el seu nom, contacte i el seu patró de compra: quantes vegades ha comprat, quant gasta de mitjana i quina va ser la seva compra rècord.

Tasca:

» **1. Crea una CTE anomenada `VIP_Stats` que agrupi per usuari i calculi:**

- La Despesa Total (SUM).
- La Quantitat de Transaccions (COUNT).
- El Tiquet Mitjà (AVG), arrodonit a 2 decimals.
- La Compra Màxima (MAX).
- Filtre: Mantén només aquells la Despesa Total dels quals sigui > 500.

» **Creua la CTE amb `users_combined` per obtenir les dades personals.**

Requisits de Sortida:

- Columnes: `user_id`, `nom_complet`, `email`, `num_compres`, `tiquet_mig`, `max_compra`, `total_gastat`.
- Ordenat per `total_gastat` descendent.

Crear la CTE:

Aunque en el enunciado no se especifica para esta columna pero sí para `Tiquet_Mitja`, considero que por legibilidad y coherencia es mejor aplicar `ROUND` también a `Despesa_Total`:

```
WITH VIP_Stats AS (  
SELECT  
    user_id,  
    ROUND(SUM(amount), 2) AS Despesa_Total,
```



```

COUNT(transaction_id) AS Quantitat_Transaccions,
ROUND(AVG(amount), 2) AS Tiquet_Mitja,
MAX(amount) AS Compra_Maxima
FROM sprint3_silver.transactions_clean
WHERE declined = 0
GROUP BY user_id
HAVING SUM(amount) > 500)

SELECT *
FROM VIP_Stats;

```

Consulta sin título Ejecutar Guardar Descargar Compartir

```

1 WITH VIP_Stats AS (
2   SELECT
3     user_id,

```

✓ Esta consulta procesará 5.91 MB cuando se ejecute.
Se está usando la cuota de procesamiento según demanda

Resultados de la consulta Chat Guardar los resultados Abrir en

Fila	user_id	Despesa_Total	Quantitat_Transa...	Tiquet_Mitja	Compra_Maxima
1	1	7612.7	25	304.51	573.13
2	2	8235.64	30	274.52	605.28
3	3	8487.36	31	273.79	650.59
4	4	7786.63	29	268.5	571.61
5	5	11120.65	38	292.65	652.54
6	6	12046.11	40	301.15	610.2

Resultados por página: 50 1 - 50 de 5000

Combinar la CTE:

Por coherencia, ya que en cada ejercicio respeto el nombre de columnas que me piden en el enunciado, realizo todos los cambios pertinentes. Por el mismo motivo que antes, mantengo ROUND (2).

```

WITH Vip_Stats AS (
  SELECT
    user_id,
    SUM(amount) AS Despesa_Total,
    COUNT(transaction_id) AS Quantitat_Transaccions,
    ROUND(AVG(amount), 2) AS Tiquet_Mitja,
    MAX(amount) AS Compra_Maxima
  FROM sprint3_silver.transactions_clean
  WHERE declined = 0
  GROUP BY user_id
  HAVING SUM(amount) > 500
)

```

```
uc.user_id,
CONCAT(uc.name, ' ', uc.surname) AS nom_complet,
uc.email,
vs.Quantitat_Transaccions AS num_compres,
vs.Tiquet_Mitja AS tiquet_mig,
vs.Compra_Maxima AS max_compra,
ROUND(vs.Despesa_Total, 2) AS total_gastat
FROM Vip_Stats AS vs
JOIN sprint3_silver.users_combined AS uc
USING (user_id)
ORDER BY total_gastat DESC;
```

Resultados de la consulta

Chat

Guardar los resultados

Abrir en

<	Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	G	>
Fila	user_id	nom_complet	email	num_compres	tiquet_mig	max_compra	total_gastat
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.39
3	455	Fwfl Phylgnmhnd	fwfl.phylgnmhnd@example.c...	66	522.59	724.44	34491.27
4	417	Fqatil Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	73	421.87	675.78	30796.3
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.79
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51

Resultados por página:

50

1 – 50 de 5000

<<

<

>

>>

Exercici 2: Anàlisi de Tendències (Window Functions sobre Vistes)

Escenari:

La Direcció Financera vol monitoritzar la "velocitat de vendes" diària. Necessiten un informe que compari el rendiment de cada dia contra el dia anterior per detectar caigudes brusques o pics de creixement (Day-over-Day Growth).

Estratègia d'Optimització:

Per a aquest càlcul NO has d'anar a la taula de fets original ni recalculiar sumes. Com a bon Data Engineer, has d'utilitzar la Vista Materialitzada `sprint3_gold.mv_daily_sales` que vas crear a l'exercici anterior. Això garanteix que l'anàlisi de tendències sigui instantània i no consumeixi recursos de còmput innecessaris.

Tasca:

Crea una consulta sobre la vista materialitzada que generi un informe amb les següents 4 columnes:

- » 1.**Data**: La data de venda.
- » 2.**Vendes_Avui**: El total venut aquell dia.
- » 3.**Vendes_Ahir**: El total venut el dia anterior (usant funcions de finestra).
- » 4.**Diff_Percentual**: El percentatge de creixement o decreixement respecte al dia anterior, arrodonit a 2 decimals.

Pista Tècnica:

Utilitza la funció `LAG(column) OVER (ORDER BY data)` per llegir el valor de la fila anterior sense necessitat de fer `JOINS` complexos.

```
WITH analisi_tendencias AS (  
  SELECT  
    Data,  
    ROUND(Vendes_Totals_Dia, 2) AS Vendes_Avui,  
    ROUND(LAG(Vendes_Totals_Dia) OVER (ORDER BY Data), 2) AS Vendes_ahir  
  FROM sprint3_gold.mv_daily_sales  
)  
  
SELECT  
  Data,  
  Vendes_Avui,
```

Vendes_Ahir,

ROUND(

SAFE_DIVIDE(

Vendes_Avui - Vendes_Ahir,

Vendes_Ahir

) * 100,

2

) AS Diff_Percentual

FROM analisi_tendencias

ORDER BY Data;

consulta lvl 2 ex 2

Ejecutar

Guardar consulta

Descargar

1 WITH analisi_tendencias AS (

2 SELECT

3 Data,

4 ROUND(Vendes_Totals_Dia, 2) AS Vendes_Avui,

5 ROUND(LAG(Vendes_Totals_Dia) OVER (ORDER BY Data), 2) AS Vendes_ahir

6 FROM sprint3_gold.mv_daily_sales

7)

8

9 SELECT

10 Data

Esta consulta procesará 57.08 KB cuando se ejecute.

consulta lvl 2 ex 2

Ejecutar

Guardar consulta

Descargar

1 WITH analisi_tendencias AS (

2 SELECT

3 Data

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Chat

Guardar los resultados

Información del trabajo

Resultados

Visualización

JSON

Fila	Data	Vendes_Avui	Vendes_Ahir	Diff_Percentual
1	2015-01-01	7360.59	null	null
2	2015-01-02	3452.21	7360.59	-53.1
3	2015-01-03	6260.01	3452.21	81.33
4	2015-01-04	6833.54	6260.01	9.16
5	2015-01-05	6978.79	6833.54	2.13
6	2015-01-06	6900.48	6978.79	-1.12

Resultados por página:

50

1 - 50

Exercici 3: Totals Acumulats (Running Totals sobre Vistes)

Escenari:

El Director Financer (CFO) necessita visualitzar la corba de creixement anual. No li interessa tant el que es va vendre avui, sinó quant portem facturat en total des de l'inici de l'any fins avui (YTD - Year To Date).

Tasca:

Utilitzant la vista materialitzada `mv_daily_sales` (per no recalcular agregacions base), genera un informe amb tres columnes:

» 1.**Data.**

» 2.**Vendes del Dia:** Arrodonit a 2 decimals.

» 3.**Vendes Acumulades YTD:** Una suma progressiva de les vendes que ha de reiniciar-se cada **1 de Gener**. El resultat final ha d'estar arrodonit a 2 decimals.

Pista Tècnica:

Per aconseguir el reinici anual, necessites dividir la finestra usant `PARTITION BY EXTRACT(YEAR FROM data)`. L'estructura completa seria: `SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`.



Consulta sin título

Ejecutar Guardar

```
1 SELECT
2   Data,
3   ROUND(Vendes_Totals_Dia, 2) AS Vendes_DeL_Dia,
4   ROUND(
5     SUM(Vendes_Totals_Dia) OVER (
6       PARTITION BY EXTRACT(YEAR FROM Data)
7       ORDER BY Data
8       ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
9     ),
10    2
11  ) AS Vendes_Acumulades_YTD
12 FROM sprint3_gold.mv_daily_sales
13 ORDER BY Data;
```

Esta consulta procesará 57.08 KB cuando se ejecute.

Información del trabajo		Resultados	Visual
Fila	Data	Vendes_DeL_Dia	Vendes_Acumulades_YTD
1	2015-01-01	7360.59	7360.59
2	2015-01-02	3452.21	10812.8
3	2015-01-03	6260.01	17072.81
4	2015-01-04	6833.54	23906.35
5	2015-01-05	6978.79	30885.14

Resultados por 1

Exercici 4: Fidelització i Valor del Client (Filtratge Avançat)

Escenari:

L'equip de Màrqueting llança la campanya "A la tercera va la vençuda". Volen enviar un regal als usuaris que hagin completat la seva tercera transacció. No obstant això, el pressupost del regal no és fix: depèn de la "qualitat" del client en els seus inicis. Necessiten saber el Tiquet Mitjà d'aquestes 3 primeres compres per segmentar si envien un detall bàsic o un regal premium.

🧩 Tasca:

Genera un llistat dels usuaris que han arribat (o superat) la seva tercera compra, mostrant:

- » 1. **Dades d'usuari (user_id, nom_complet, email).**
- » 2. **La data i l'import exacte de la 3a compra.**
- » 3. **Mitjana de les 3 primeres:** La mitjana de despesa de les seves transaccions 1, 2 i 3.

💡 Pista Tècnica:

- » **Has d'utilitzar la funció ROW_NUMBER() per establir l'ordre cronològic de les compres.**
- » **Has d'utilitzar la clàusula QUALIFY per filtrar.**
- » **Repte d'Optimització:** Busca l'estratègia de menor cost computacional. Tingues en compte que calcular mitjanes sobre tot l'historial històric és costós. Intenta filtrar les files necessàries abans de realitzar l'agregació final.

```
WITH tres_compres AS (  
  SELECT  
    tc.user_id,  
    uc.name,  
    uc.surname,  
    uc.email,  
    tc.timestamp AS data_compra,  
    tc.amount AS import,  
    ROW_NUMBER() OVER (  
      PARTITION BY tc.user_id  
      ORDER BY tc.timestamp  
    ) AS rn
```

```

FROM sprint3_silver.transactions_clean tc
JOIN sprint3_silver.users_combined uc
    USING (user_id)
)

SELECT
    user_id,
    CONCAT(name, ' ', surname) AS nom_complet,
    email,
    data_compra AS data_tercera_compra,
    import AS import_tercera_compra,
    AVG(import) OVER (PARTITION BY user_id) AS mitjana_3_primers
FROM tres_compres
QUALIFY rn <= 3;

```

1 WITH tres_compres AS (
2 SELECT
3 tc.user_id,
4 uc.name,
5 uc.surname,
6 uc.email,
7 tc.timestamp AS data_compra,
8 tc.amount AS import,
9 ROW_NUMBER() OVER (
10 PARTITION BY tc.user_id

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta [Chat](#) [Guardar los resultados](#) [Abrir en](#)

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	user_id	nom_complet	email	data_tercera_compra	import_tercera_c...	m
1	1	Zeus Gamble	interdum.enim@protonmail.edu	2015-05-08 18:09:41 UTC	233.52	3
2	1	Zeus Gamble	interdum.enim@protonmail.edu	2015-11-04 13:01:34 UTC	322.53	3
3	1	Zeus Gamble	interdum.enim@protonmail.edu	2015-12-20 18:06:59 UTC	161.33	3
4	2	Garrett Mcconnell	integer.vitae.nibh@protonmail.o...	2015-09-21 01:23:22 UTC	267.6	2

Resultados por página: 50 1 – 50 de 15000

Utilizó QUALIFY en vez de MAX/AVG con IF porque permite filtrar directamente el resultado de la “window function” sin necesidad de hacer otras subconsultas, lo que facilita la lectura y simplifica la query.

Para reducir costos y así optimizar, limitamos el conjunto de datos a las 3 primeras compras (transacciones) por usuario antes de calcular la media.

Nivell 3: Analytics Engineering (Arrays & Automatització)

Escenari:

Màrqueting necessita urgentment un Rànquing de Productes més Venuts.

Exercici 1: Desanidament i Aplanament de Dades (Unnesting)

Escenari:

Actualment, a la taula de transaccions, els productes venuts estan "amagats" dins d'un Array (una llista d'IDs). L'equip de vendes vol analitzar quins productes específics es venen més, però no poden fer-ho si les dades estan comprimides en una llista. Necessiten una Taula Detallada (Flat Table) on cada fila representi un producte venut, encara que això signifiqui que l'ID de la transacció es repeteixi.

🧩 Tasca:

Crea la taula `dim_transactions_flat` desnormalitzant la informació. Has d'"explotar" l'array de productes i creuar-lo amb el catàleg mestre per obtenir els noms i preus individuals.

⚙️ Passos Tècnics:

- » 1. Utilitza `CROSS JOIN UNNEST(product_ids)` per transformar l'Array en files individuals.
- » 2. Realitza un `JOIN` amb la taula `products` per obtenir el nom (`name`) i preu (`price`) de cada ítem.

📌 *Nota: Ves amb compte amb els tipus de dades en fer el `JOIN` (Array de `INT64` vs `Product_ID STRING`).*

- » 3. No agrupeu el resultat. Volem veure el desglossament línia per línia.

✅ Resultat Esperat (Visualització):

La teva taula final ha de veure's "plana", on una transacció amb 3 productes ocupa 3 files:

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price

TXN1001	2025-12-08	155.97	SKU_A	Portàtil Bàsic	99.99
TXN1001	2025-12-08	155.97	SKU_B	Ratolí	12.99
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclat	45.50

👉 Entrega:

- » Una captura de la Vista Prèvia de la teva taula on es vegi el DDL executada correctament.
- » Una captura de pantalla on es vegi l'esquema de la nova taula.

```

1 CREATE OR REPLACE TABLE sprint3_gold.dim_transactions_flat AS
2 SELECT
3   tc.transaction_id,
4   tc.user_id,
5   product_id,
6   pc.name,
7   pc.price
8 FROM sprint3_silver.transactions_clean AS tc
9 CROSS JOIN UNNEST(tc.product_ids) AS product_id
10 JOIN sprint3_silver.products_clean AS pc
11   ON CAST(product_id AS INT64) = pc.product_id;
12

```

✓ Esta consulta procesará 6.32 MB cuando se ejecute.

```

1 CREATE OR REPLACE TABLE sprint3_gold.dim_transactions_flat AS
2 SELECT
3   tc.transaction_id,
4   tc.user_id,
5   product_id,
6   pc.name AS product_name,
7   pc.price AS product_price,
8 FROM sprint3_silver.transactions_clean AS tc
9 CROSS JOIN UNNEST(tc.product_ids) AS product_id
10 JOIN sprint3_silver.products_clean AS pc
11   ON CAST(product_id AS INT64) = pc.product_id;

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

🔍 Filtro Escribir el nombre o valor de la propiedad

☐	Nombre del campo	Tipo	Modo
☐	transaction_id	STRING	NULLABLE
☐	user_id	INTEGER	NULLABLE
☐	product_id	INTEGER	NULLABLE
☐	product_name	STRING	NULLABLE
☐	product_price	FLOAT	NULLABLE

Observación: Al realizar el ejercicio 3 de este nivel, vemos que es necesario modificar esta tabla añadiendo `tc.timestamp` al `SELECT`, para poder usar esa dimensión en el gráfico.

Exercici 2: El Rànquing de Vendes (Agregació Simple)

Escenari:

Ara que has preparat la infraestructura i disposes de la taula `dim_transactions_flat` (on cada fila equival a un producte venut), generar informes analítics és trivial. Ja no necessites funcions complexes de desanidament per respondre preguntes bàsiques de negoci.

✂ Tasca:

Genera el **Top 5 de productes** més venuts en la història de la companyia.

🔧 Mètode:

Consulta directament la teva nova taula desnormalitzada (`dim_transactions_flat`).

» En ser la taula plana, simplement necessites una agrupació estàndard (`GROUP BY`) pel nom del producte i un compte (`COUNT`) de les files.

👉 Entrega:

- » El codi SQL que genera el rànquing.
- » Una captura del resultat amb els 5 productes guanyadors i quantes unitats s'han venut de cadascun.

```
1 SELECT
2   product_name,
3   COUNT(*) AS unitats_venudes
4 FROM sprint3_gold.dim_transactions_flat
5 GROUP BY product_name
6 ORDER BY unitats_venudes DESC
7 LIMIT 5;
```

✅ Esta consulta procesará 4.06 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo		Resultados	Visualización
Fila	product_name	unitats_venudes	
1	Winterfell	10071	
2	the duel	10059	
3	dooku solo	10022	
4	Karstark Dorne	7694	
5	skywalker ewok	7607	

Exercici 3: Automatització del Pipeline i Visualització

Escenari:

L'informe estàtic de l'exercici anterior ha agradat molt, però el Director de Màrqueting necessita professionalitzar-lo per a la presa de decisions diària. Té dos requeriments urgents:

- » **1. Lògica Centralitzada:** Volen veure el preu amb IVA (21%) desglossat. No obstant això, com que l'impost pot canviar per país o any, et prohibeixen posar el càlcul $\text{preu} * 1,21$ directament en el codi ("Hardcoded"). Has d'utilitzar una funció reutilitzable.
- » **2. Frescor de Dades:** El tauler ha d'estar actualitzat automàticament cada matí a les 07:00 AM, abans de la reunió diària de vendes.

🧩 Tasca:

Has d'evolucionar la teva taula `dim_transactions_flat` per incloure el càlcul d'impostos i automatitzar la seva regeneració.

⚙️ Passos Tècnics:

- » **1. User Defined Functions (UDF):** Crea una funció SQL persistent anomenada `calculate_tax(amount)` que rebi un valor numèric i retorni el resultat aplicant el 21% d'impost.
- » **2. Integració i Orquestració:**
 - Modifica el codi de creació de la taula (de l'Exercici 1) perquè utilitzi la teva nova funció `calculate_tax` i generi una columna nova: `product_price_tax_inc`.
 - Configura una Scheduled Query a BigQuery perquè aquesta sentència (CTAS) s'executi automàticament cada dia a les 07:00 AM.
- » **3. Visualització (BI):** Connecta Looker Studio a la teva taula `dim_transactions_flat` i crea un Dashboard: "Monitor de Rendiment de Vendes"

👉 Entrega:

- » El codi SQL de la UDF.
- » El codi SQL actualitzat de la creació de la taula.
- » Una captura de pantalla demostrant que la Scheduled Query està programada.

» Un enllaç del gràfic a Looker Studio.

Proposta de Dashboard:

"Monitor de Rendiment de Vendes"

Configura una pàgina amb 3 seccions clau:

» 1. Els "Big Numbers" (KPIs Generals):

A la part superior, col·loca tres targetes de resultats (Scorecards) per donar context immediat.

- **KPI 1: Ingressos Totals (Amb IVA)**
 - **Mètrica:** `SUM(product_price_tax_inc)`
 - *Per què:* Mostra el volum real de diners que entra.
- **Transaccions Úniques (Comandes)**
 - **Mètrica:** `COUNT_DISTINCT(transaction_id)`
- **Transaccions Úniques (Comandes)**
 - **Camp Calculat:** Crea un camp a Looker:
`SUM(product_price_tax_inc) /`
`COUNT_DISTINCT(transaction_id)`

» 2. Evolució Temporal (Gràfic de Sèries Temporals)

Sota els KPIs, un gràfic de línia que ocupi tot l'ample.

- **Dimensió:** timestamp (Desglossat per Data).
- **Mètrica:** `SUM(product_price_tax_inc)`.
- **Línia de Referència (Opcional):** Afegeix una mitjana constant o una línia de tendència.
- **Valor:** Permet a Màrqueting veure si les vendes pugen o baixen dia a dia, o si hi va haver un pic per alguna campanya específica.

» El "Top Productes" Enriquit (Taula amb Mapa de Calor)

Oblida't del gràfic de barres simple. Utilitza una Taula amb Barres incrustades o Mapa de Calor.

- **Dimensió:** `product_name`.
- **Mètrica 1:** `COUNT(*)` → Renombra-ho a "Unitats Venudes".
- **Mètrica 2:** `SUM(product_price_tax_inc)` → Renombra-ho a "Ingressos Totals".
- **Mètrica 3:** `SUM(product_price_tax_inc) -`
`SUM(product_unit_price)` → Renombra-ho a "IVA Recaptat" (Això demostra el valor de la teva UDF).
- **Estil:** A la configuració d'estil de la taula, activa l'opció "Mostra número com a: Barra" o "Mapa de calor" per a la columna d'Ingressos.

El codi de la UDF que he ejecutado primero:

```
CREATE OR REPLACE FUNCTION sprint3_gold.calculate_tax(amount FLOAT64)
RETURNS FLOAT64 AS (
  amount * 1.21
);
```

El codi SQL actualitzat de la creació de la taula:

```
CREATE OR REPLACE TABLE sprint3_gold.dim_transactions_flat AS
SELECT
  tc.transaction_id,
  tc.user_id,
  product_id,
  pc.name,
  pc.price,
  sprint3_gold.calculate_tax(pc.price) AS product_price_tax_inc
FROM sprint3_silver.transactions_clean AS tc
CROSS JOIN UNNEST(tc.product_ids) AS product_id
JOIN sprint3_silver.products_clean AS pc
  ON product_id = CAST(pc.product_id AS INT64);
```

Programar la query:

consulta lvl 3 ex 3 [Ejecutar](#) [Guardar consulta](#) [Descargar](#) [Compartir](#) [Programa](#)

Exportar como consulta programada

✓ Configuración válida.

Detalles y programación

Nombre de la consulta programada *
dim_transactions_flat_actualizada

Opciones de programación

Frecuencia de repetición *
Días

A la(s) *
7:00 UTC

☐ Comenzar ahora ☒ Comenzar a la hora definida

Fecha de inicio y hora de ejecución *
10/6/26, 3:50 p.m. 000

☒ Sin hora de finalización programada
☐ Programar una hora de finalización

[Guardar](#) [Cancelar](#)

Enllaç al Dashboard:

<https://datastudio.google.com/reporting/33db594c-b55e-4108-b084-0361b3e9ed8c>

Vista prèvia:

(hay un error en el enunciado, la 2a y 3a scorecards las plantea con el mismo título, así que decido cambiarlo para mi Dashboard para que el título sea más representativo)

